VOGUE: VirtIO-GPU on Unikraft

Current-stage implementation and evaluation

Graphics and llama.cpp inside Unikraft

2026-06-04

Outline

Outline

Motivation

Unikernels: tiny, instant, secure — and headless

A **unikernel** fuses a single application with just-enough OS into one bootable image: one address space, no processes, booted straight by the hypervisor.

Unikernels: tiny, instant, secure — and headless

A **unikernel** fuses a single application with just-enough OS into one bootable image: one address space, no processes, booted straight by the hypervisor.

Size

image in **hundreds of KB**, not hundreds of MB

Boot

ready in **milliseconds**, not seconds

Security

small attack surface — only the code linked

Unikernels: tiny, instant, secure — and headless

A **unikernel** fuses a single application with just-enough OS into one bootable image: one address space, no processes, booted straight by the hypervisor.

Size

image in **hundreds of KB**, not hundreds of MB

Boot

ready in **milliseconds**, not seconds

Security

small attack surface — only the code linked

- every one of these wins comes from leaving things out
  - no shell, no user/kernel separation, no multi-process...
  - the ecosystem is still young; many drivers and features arn not ported yet

GPU workloads are real — Unikraft can’t run them yet

Plenty of in-demand workloads need a GPU:

- **Graphics / rendering** — desktop & GUI apps, 3D visualisation, games
- **LLM inference** — e.g. `llama.cpp`, a popular open-source inference engine
- **...and more** — ML training, video transcode, scientific compute

GPU workloads are real — Unikraft can't run them yet

Plenty of in-demand workloads need a GPU:

- **Graphics / rendering** — desktop & GUI apps, 3D visualisation, games
- **LLM inference** — e.g. `llama.cpp`, a popular open-source inference engine
- **...and more** — ML training, video transcode, scientific compute

But Unikraft has no GPU support today

- Those workloads either can't run at all, or have to run on the CPU
- Solution: *add GPU support to Unikraft*

The dilemma — and the question we ask

The standard way to give a guest GPU access (`virtio-gpu` + Mesa Venus) requires the **entire Linux graphics stack** in the guest: the kernel DRM / KMS subsystem, `libdrm` , Mesa, and more.

That is exactly the bulk a unikernel exists to avoid. Getting the GPU looks like it means "become Linux again" .

The dilemma — and the question we ask

The standard way to give a guest GPU access (`virtio-gpu` + Mesa Venus) requires the **entire Linux graphics stack** in the guest: the kernel DRM / KMS subsystem, `libdrm` , Mesa, and more.

That is exactly the bulk a unikernel exists to avoid. Getting the GPU looks like it means "become Linux again" .

Can a *minimal* unikernel guest reach a GPU without importing Linux's graphics stack?

The dilemma — and the question we ask

The standard way to give a guest GPU access (virtio-gpu + Mesa Venus) requires the **entire Linux graphics stack** in the guest: the kernel DRM / KMS subsystem, libdrm , Mesa, and more.

That is exactly the bulk a unikernel exists to avoid. Getting the GPU looks like it means "become Linux again".

Can a *minimal* unikernel guest reach a GPU without importing Linux's graphics stack?

- Concretely: run GPU-accelerated workloads (graphics, LLM inference) **without giving up** the unikernel's tiny image and instant boot

Outline

Background

Unikraft: the library OS we build on

Unikraft is a framework that builds a unikernel from **modular libraries**:

- you pick exactly what you need
  - a libc (`musl`), a TCP/IP stack (`lwIP`), a scheduler, device drivers, etc.
- only those are linked with your one app into a single image, booted by QEMU/KVM.

Unikraft: the library OS we build on

Unikraft is a framework that builds a unikernel from **modular libraries**:

- you pick exactly what you need
  - a libc (`musl`), a TCP/IP stack (`lwIP`), a scheduler, device drivers, etc.
- only those are linked with your one app into a single image, booted by QEMU/KVM.

What you get

- one address space, one purpose per image
- only the libraries you asked for are linked in

What is not there

- no Linux kernel — no `/dev/dri` , no kernel modules
- no processes, no `fork` / `exec` , no dynamic loader

Unikraft: the library OS we build on

Unikraft is a framework that builds a unikernel from **modular libraries**:

- you pick exactly what you need
  - a libc (`musl`), a TCP/IP stack (`lwIP`), a scheduler, device drivers, etc.
- only those are linked with your one app into a single image, booted by QEMU/KVM.

What you get

- one address space, one purpose per image
- only the libraries you asked for are linked in

What is not there

- no Linux kernel — no `/dev/dri` , no kernel modules
- no processes, no `fork` / `exec` , no dynamic loader

- Consequence: anything Linux hands you for free — device nodes, mapping GPU memory, loading a driver — we must **build ourselves, or do without**

Why we use virtio-gpu

Two roads exist for virtualizing GPUs: GPU passthrough or para-virtual **virtio-gpu**. We take virtio-gpu, for two reasons that matter to a **tiny unikernel** specifically:

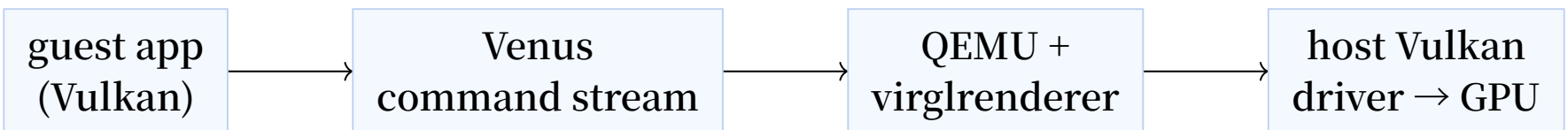
- A natural fit for tiny VMs:
  - one physical GPU **shared** across many guests, especially for lightweight workloads like GUI apps
  - the guest needs only a **thin frontend**, so the image stays unikernel-tiny and migratable
- Cheap and thin to implement:
  - just a **standard virtio device** — Unikraft already has the virtio / PCI transport; we reuse it unchanged

virtio-gpu and Venus: shipping GPU commands to the host

- **virtio-gpu** — the paravirtual GPU device: virtio queues, GPU resources, fences
- **Venus** (from Mesa) — don’t translate graphics; serialize the Vulkan API itself into a command stream, ship it over `virtio-gpu`, and let the host replay it on its real Vulkan driver

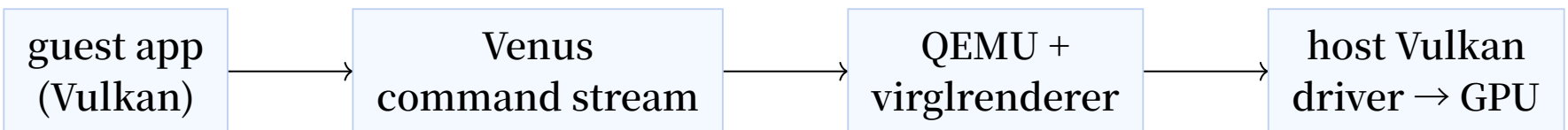
virtio-gpu and Venus: shipping GPU commands to the host

- **virtio-gpu** — the paravirtual GPU device: virtio queues, GPU resources, fences
- **Venus** (from Mesa) — don't translate graphics; serialize the Vulkan API itself into a command stream, ship it over `virtio-gpu`, and let the host replay it on its real Vulkan driver



virtio-gpu and Venus: shipping GPU commands to the host

- **virtio-gpu** — the paravirtual GPU device: virtio queues, GPU resources, fences
- **Venus** (from Mesa) — don't translate graphics; **serialize the Vulkan API itself** into a command stream, ship it over `virtio-gpu`, and let the host replay it on its real Vulkan driver



- the **modern** path, and the one we target
- the guest only **emits commands** — the real GPU work happens host-side

What this costs a Linux guest: the tower

On Linux, one Vulkan call falls through a **tall stack** before it ever leaves the guest:

Layer (top → device)	Owned by…
application	the app
Vulkan loader	**Linux graphics stack** four guest-side layers, none of which a unikernel wants
Mesa Venus ICD (userspace driver)	
libdrm	
kernel DRM virtgpu driver (+ DRM / KMS / GEM)	
virtio-gpu device	the hypervisor

What this costs a Linux guest: the tower

On Linux, one Vulkan call falls through a **tall stack** before it ever leaves the guest:

Layer (top → device)	Owned by…
application	the app
Vulkan loader	**Linux graphics stack** four guest-side layers, none of which a unikernel wants
Mesa Venus ICD (userspace driver)	
libdrm	
kernel DRM virtgpu driver (+ DRM / KMS / GEM)	
virtio-gpu device	the hypervisor

- but what **crosses to the host** is only a stream of `VIRTIO_GPU_CMD_*` + Venus commands
- Key seam: the host doesn't care **who** emits them — same stream, interchangeable guests
- so a guest can speak this protocol with a **fraction** of the tower (← that is our Design)

Outline

System Design

Key Principle: Single-Purpose, Single-Client GPU Driver

Unikraft = one process, one purpose → GPU has exactly one client

Replace DRM multiplexing with a **minimal exclusive-client driver**

Dimension	Linux virtio-gpu	VOGUE
Graphics stack	3M LoC	5K LoC
Address space	Kernel + User	Single
GPU clients	Multiple (needs arbiter)	Exclusive single client
Command path	app → ioctl → kernel → virtio	app → direct virtio ring

Key Principle: Single-Purpose, Single-Client GPU Driver

Unikraft = one process, one purpose → GPU has exactly one client

Replace DRM multiplexing with a **minimal exclusive-client driver**

Dimension	Linux virtio-gpu	VOGUE
Graphics stack	3M LoC	5K LoC
Address space	Kernel + User	Single
GPU clients	Multiple (needs arbiter)	Exclusive single client
Command path	app → ioctl → kernel → virtio	app → direct virtio ring

Unlocks:

- No DRM multiplexing → 1,650 LoC driver
- No user/kernel boundary → virtqueue writes are direct function calls, no ioctl

VOGUE System Architecture

Layer	Components
L1 Application	kmscube / glmark2 / llama.cpp (**unmodified**)
L2 Compat	libukegl + libukswrender (2D display) · libukggml_vk (Vulkan dispatch)
L3 Venus	libukvenus (Vulkan → Venus binary)
L4 VirtIO	libukvirtio_gpu + libukdma (→ Unikraft virtqueue → QEMU)

VOGUE System Architecture

Layer	Components
L1 Application	kmscube / glmark2 / llama.cpp (**unmodified**)
L2 Compat	libukegl + libukswrender (2D display) · libukggml_vk (Vulkan dispatch)
L3 Venus	libukvenus (Vulkan → Venus binary)
L4 VirtIO	libukvirtio_gpu + libukdma (→ Unikraft virtqueue → QEMU)

- Host: `QEMU virtio-gpu-gl-pci` → `virglrenderer` → Host Vulkan / GL driver
- Guest image size: **292 KB** (vs Linux kernel 9.2 MB — 31× smaller)
- Boot time: **10–11 ms** (vs Linux + initramfs 857–861 ms — 80× faster)

VOGUE vs Linux: Layer by Layer

	Linux VM		VOGUE Unikraft
L1 App	**app** kmscube / llama.cpp	≡	**app** unmodified upstream
L2 Compat	**libEGL + libgbm + libdrm + Vulkan loader** real Linux platform APIs	→	**libukegl · libukswrender · libukggml_vk** stub + CPU render + static Vk dispatch
L3 Venus	**Mesa Venus driver** dynamic, Vulkan→Venus encoding	→	**libukvenus** same encoding, no Mesa
L4 VirtIO	**virtio-gpu + kernel virtio/PCI** /dev/dri ioctl → kernel → virtqueue	→	**libukvirtio_gpu** direct virtqueue, no ioctl

QEMU virtio-gpu → virglrenderer → Host GPU driver *(shared)*

≡ same · → replaced with leaner VOGUE equivalent

Three Execution Paths

	2D Display	3D Rendering	Vulkan Compute
L1 App	**app** kmscube SW · glmark2	**app** kmscube virgl	**app** llama.cpp
L2 Compat	**libukswrender** CPU rasterizer	— direct virgl_encoder call	**libukggml_vk** static Vk dispatch
L3 Venus	— no Vulkan, CPU path only	— virgl has no L3 serializer	**libukvenus** Vulkan → Venus binary
L4 VirtIO	**TRANSFER_TO_HOST_2D** SET_SCANOUT · FLUSH	**SUBMIT_3D** virgl_encoder.c in L4	**SUBMIT_3D** Venus payload
GPU	none CPU only	**host GPU** → buffer	**host GPU** → buffer (no display)

Design Goals

- **DG1** Standards-based: follow the OASIS VirtIO-GPU spec, compatible with QEMU / crosvm / cloud-hypervisor
- **DG2** Small trusted substrate: guest graphics stack 5,000 LoC, fully auditable
- **DG3** Application source compatibility: kmscube / llama.cpp upstream sources **unmodified**
- **DG4** Progressive validation: bottom-up — native tests → SW render → ABI → QEMU transport → accelerated render

Design Goals

- **DG1** Standards-based: follow the OASIS VirtIO-GPU spec, compatible with QEMU / crosvm / cloud-hypervisor
- **DG2** Small trusted substrate: guest graphics stack 5,000 LoC, fully auditable
- **DG3** Application source compatibility: kmscube / llama.cpp upstream sources **unmodified**
- **DG4** Progressive validation: bottom-up — native tests → SW render → ABI → QEMU transport → accelerated render

DG5 — Evidence-Gated Claims

- `gfx.kmscube.sw` PASS ≠ virgl / GPU acceleration
- `proto.real-driver` PASS ≠ QEMU execution
- Host Linux baseline ≠ Unikraft runtime

Outline

Implementation I Driver Stack

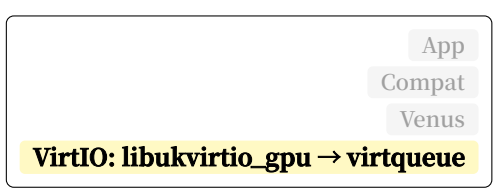
VirtIO-GPU: The Core Driver

libukvirtio_gpu — full VirtIO-GPU protocol in 1,650 LoC, exclusive device ownership

2D display pipeline (enables kmscube, glmark2):

CPU draws pixels → push to QEMU → bind to display → wait

Init once: allocate host framebuffer, link to guest memory



VirtIO-GPU: The Core Driver

libukvirtio_gpu — full VirtIO-GPU protocol in 1,650 LoC, exclusive device ownership

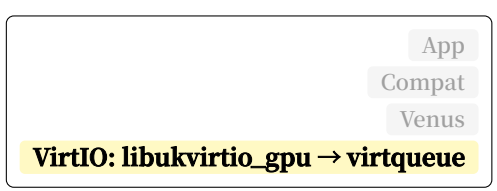
2D display pipeline (enables kmscube, glmark2):

CPU draws pixels → push to QEMU → bind to display → wait

Init once: allocate host framebuffer, link to guest memory

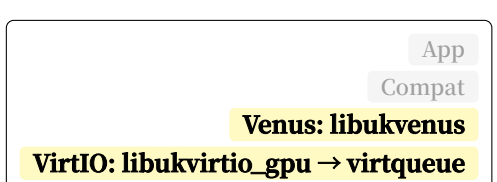
Insight: VirtIO-GPU executes commands in order

→ waiting on "present" already implies "transfer" is done
→ **one fence wait instead of two per frame**



VirtIO-GPU Venus: GPU Acceleration

Venus extends VirtIO-GPU: Vulkan API calls serialized into binary → `SUBMIT_3D` → host GPU



VirtIO-GPU Venus: GPU Acceleration

Venus extends VirtIO-GPU: Vulkan API calls serialized into binary → `SUBMIT_3D` → host GPU

Two mechanisms working together:

Shared memory (L4)

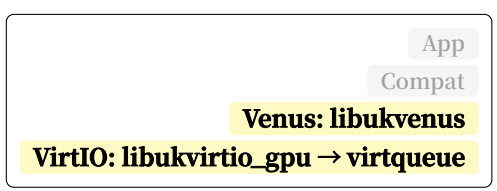
- Guest and host map the **same** physical memory
- Guest writes, host reads — **zero copy**
- No DMA transfer needed

Ring buffer (L3)

- Vulkan calls batched in shared memory
- Host notified **once** per batch
- Amortizes SUBMIT_3D cost

Result: virgl pixel-correct frame ✓ · llama.cpp GPU at 160 tok/s ✓

Protocol encoding verified with a software-only fake backend · end-to-end results on real QEMU + host GPU



Outline

Implementation II Applications

Applications: small ports, real workloads

App	Role	VOGUE glue	What it proves
kmscube	minimal graphics demo	593 LOC / 3 ABI shims	unchanged app draws inside our Unikraft system
glmark2	OpenGL app bring-up	155 LOC / 3 ABI shims	a larger GL app can start and run on our path
vkmark	Vulkan app bring-up	62 LOC / 2 Vulkan shims	a Vulkan app can start and load its scenes
llama.cpp	real compute workload	upstream bridge + dispatch	LLM inference runs inside Unikraft

- The application code stays upstream; VOGUE provides the missing system pieces.
- LOC here is VOGUE-owned glue, not the vendored application body.

Issue: Graphics has more moving parts

Compute pipeline	Graphics pipeline
one main object: `VkComputePipelineCreateInfo`	many fixed-function state blocks
one shader stage: compute	at least vertex + fragment stages
data in, data out	vertices, viewport, rasterization, depth/stencil, blending, presentation
fits one-purpose appliances	expects a window/display stack

Compute is a shorter path. Graphics needs many more pieces around the shader work before anything appears on screen.

Compute was the practical first target

What we compare	Compute pipeline	Graphics pipeline
Pipeline setup	one create struct	VkGraphicsPipelineCreateInfo plus 10+ state structs
Shader stages	one compute shader	vertex + fragment shaders at minimum
Work flow	one short execution path	many linked stages must all match
Screen output	no screen output needed	must manage render targets and show images on screen

In Vulkan, compute gave us one smaller path to get working first. Graphics required a larger setup before we could show real images.

- After proving the graphics basics, compute was the faster path to a real GPU workload.

Current Stage for Graphics Applications

App / row	Current
kmscube.sw	software-rendered graphics works
kmscube.submit/frame	virgl sends SUBMIT_3D and we captured the clear frame
glmark2.sw	the scene-clear run works on our software path
vkmark	the Vulkan app starts and loads scenes

Each row here means a different thing. Some rows prove software rendering, some prove transport, and some prove real GPU compute.

The compute path reaches the real GPU

upstream llama.cpp bench/server
→ ggml-vulkan
→ libukggml_vk static dispatch
→ libukvenus encoder + Venus protocol
→ libukvirtgpu_drm + libukvirtio_gpu
→ QEMU virtio-gpu-gl + virglrenderer
→ host Vulkan driver / Tesla V100

Appliance	Current status
CPU bench	baseline inference passes
CPU server	readiness appliance passes
Vulkan bench	GPU inference over Venus passes
Vulkan HTTP	real requests over lwIP pass

Library	Role	Source LOC
libukggml_vk	static ggml-vulkan dispatch	2,146
libukvenus	mostly generated Venus encoder/protocol + ring glue	96,690
libukvirtgpu_drm	Linux virtgpu DRM ioctl shim	638
libukvirtio_gpu	VirtIO-GPU frontend + virgl/controlq	2,246
libukvk_icd	Vulkan ICD bootstrap shim	235

- `libukvenus` is mostly **generated** from upstream Venus protocol and Vulkan API metadata.
- The hand-written part is the Unikraft-side ring and glue code.

Outline

Evaluation

Evaluation host: one real GPU path

Layer	Setup
VM monitor	QEMU 11.0.1
GPU device	virtio-gpu-gl-pci, blob=true, venus=true
Host bridge	Venus-enabled virglrenderer
Host GPU	Tesla V100-SXM2-16GB
Guest	Unikraft appliance, one image per purpose
Rule	same-run artifact or no claim

We evaluate the same bridge Linux uses, but with a much thinner Unikraft guest.

Current stage

What we checked	Goal	Status today
Graphics basics	graphics apps can open buffers and draw	virgl sends 3 real 3D submits in our small test
Vulkan basics	create devices, talk to Venus, and start a real app	DRM shim, ICD setup, Venus command path, and vkmark scene loading all run
llama.cpp GPU	upstream compute uses the real GPU through Venus	ggml-vulkan inference runs inside Unikraft on the Tesla V100
llama.cpp HTTP	GPU workload is usable as a service	endpoint `/health` lwIP

Graphics results today

Workload	Current number / result	What we can say
glmark2.sw	142.82 fps in the 120-frame scene-clear run	subset benchmark, not full suite
kmscube virgl	3 virgl-submitted frames and 3 SUBMIT_3D commands	one small clear-frame proof
vkmark	10 startup scenes load through our Vulkan app path	no Unikraft FPS yet

Graphics already runs in our system, but we still separate software rendering, app bring-up, and full benchmark claims.

llama.cpp: compute works, performance gap remains

Same model / GPU path	Prefill pp512	Generate tg128
CPU Unikraft	9.4	7.4
VOGUE Unikraft + Venus	2045.8	139.3
Linux VM + Venus	4948.17	323.64
Bare-metal Vulkan	5586.91	239.16

`pp512` = 512-token prompt ingestion speed. `tg128` = 128-token decode speed. Units: tokens/s.

- GPU offload is a large jump over CPU-only Unikraft.
- Linux VM over the same Venus bridge is still faster.
- The remaining performance difference is in our Vulkan library and driver code.

The server works

Server fact	Current result
HTTP liveness	/health , /v1/models , /completion return 200
Slots configured	4
Throughput run	8 requests at concurrency 1
Prompt throughput	146.14 tokens/s during prompt ingestion
Decode throughput	25.53 tokens/s during token generation
TTFT	4.98 s to first token
Request rate	0.159 requests/s in this small burst test

The server now works. The next step is making it faster under real load.

Outline

Future work

From working prototype to performance

Priority	Next step	Why it matters
P0	Add guest SMP and more vCPUs	today we only measured the server at concurrency 1
P0	Tune llama server threads and slots	move from “it works” to “it can handle real load”
P1	Reduce time spent in our Vulkan call path	Linux VM on the same Venus bridge is still faster
P1	Load the model with less copying	today the model path still runs without `mmap` or huge pages
P1	Add matched Linux graphics baselines	make graphics results a fair comparison

- These are not old bring-up blockers; they are the next performance and breadth targets.

Where the next speedup should come from

Bottleneck signal	Current measurement	Planned attack
Linux VM + Venus beats VOGUE	323.64 vs 139.3 `tg128` decode t/s	reduce time spent in our Vulkan call path
server throughput is low	25.53 decode t/s in the 8-request burst	add SMP and split server threads
model loading copies too much	Vulkan server load is 5935.26 ms with `use_mmap=false`	add an mmap-capable model path
graphics lacks broad scene FPS	vkmark loads 10 scenes but has no Unikraft FPS	add small scene-render tests and frame checks

We now know where to look: in our own Vulkan and server code, not in a missing GPU bridge.

Long-term: reusable GPU unikernel method

- Keep turning the VirtIO-GPU / Venus stack analysis into reusable guest libraries
- Keep the one-image-one-purpose app ports reproducible
- Extend from today’ s demos and llama.cpp server to more graphics and GPU workloads

The long-term value is not one benchmark number; it is a reusable way to understand the GPU stack, port applications, and keep the guest small.

Outline

Conclusion

Three takeaways

- We analyzed the GPU path
  - VOGUE breaks down VirtIO-GPU into driver, library, graphics, and compute pieces
- We showed a thin Unikraft design
  - small guest libraries reach display, Vulkan, and Venus without importing Linux
- We proved real workloads
  - we ported apps, ran graphics demos, and served llama.cpp over the real GPU

Our contribution is not just one demo: we studied the GPU stack, built the Unikraft libraries we needed, and ran real applications plus the llama.cpp server.

The End!

Thank you!